0. **Code:** Dahiya ISAM5430 Final.zip

1. Dahiya ISAM5430 Final / A / A / Program.cs
2. Dahiya ISAM5430 Final / B. sorting / B. sorting / Program.cs
3. Dahiya ISAM5430 Final / C. Dictionary / C. Dictionary / Program.cs

1. Dahiya ISAM5430 Final / A / A / Program.cs

```
namespace A_
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Enter an integer:");
            int num = Convert.ToInt32(Console.ReadLine());
            Console.WriteLine("Enter base-N");
            int n = Convert.ToInt32(Console.ReadLine());
            int temp = 0;
            int temp2 = 0;
            int sum = 0;
            int d = num / n;
            int rem, rem2;

            while (true)
            {
                if (n < 0)
                {
                    Console.WriteLine("Enter base-N (must be positive)");
                    continue;
                }
                rem = num % n;
                rem2 = d % n;
                sum = rem + rem2;
                Console.WriteLine(rem2);
            }
```

A1

int n = -1;
while (n ≤ 0){
}

What?

A2

Separate loop.

if(num<0){
num = -num;
}

(num>0)

loop

num=num/n;

A3 X

sum =sum+ rem;

A4 X

A5 Did not attempt to find the smallest.

X

1. Dahiya ISAM5430 Final / A / A / Program.cs

```
        }
    }
}
```

2. Dahiya ISAM5430 Final / B. sorting / B. sorting / Program.cs

```
namespace B._sorting
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int[] array = {
                25,
                1,
                10,
                5,
                2
            };
            Console.WriteLine(string.Join(",", array));
        }
        public static int[] vd(int[] array)
        {
            int[] count = new int[array.Length];
            for (int i = 0; i < count.Length - 1; i++)
            {
                for (int j = i + 1; j > 0; j--)
                {
                    if (count[j - 1] > count[j])
                    {
                        int temp = count[j - 1];
                        count[j - 1] = count[j];
                        count[j] = temp;
                    }
                }
            }

```

B1

all 0's

Inconsistent with Insertion Sort!

always false

j<=i j++

What? Swapping

2. Dahiya ISAM5430 Final / B. sorting / B. sorting / Program.cs

```
            return count;
        }
    }
}
```

```
namespace C._Dictionary
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int[] array = {
                1,
                2,
                3,
                2,
                7,
                3,
                4,
                4,
                5,
                5,
                4,
                5,
                1,
                6,
                1,
                1
            };
            Console.WriteLine(string.Join(",", array));
        }
        public static int[] vd(int[] array)
        {
            if (array == null)
            {
                return 0;
```

```
            }
            var dict = new Dictionary < int,
                int > ();
            foreach(int v in array)
            {
                if (!dict.ContainsKey(v))
                {
                    dict.Add(v, 1);
                }
                else
                {
                    dict[v]++;
                }
                int max = int.MinValue;
                var dict2 = new Dictionary < int,
                    int > ();
                foreach(int kvp in array)
                {
                    if (!dict2.ContainsKey(kvp))
                    {
                        dict2.Add(kvp, 1);
                    }
                    else
                    {
                        dict2[kvp]++;
                    }
                }
            }
            int[] result = new int[array.Length];
            int result_index = 0;
            foreach(var kvp in dict)
```

```
            {
                int count = kvp.Value;
                while (count > 0)
                {
                    result[result_index] = kvp.Key;
                    count--;
                    result_index++;
                }
            }

            return result;
        }


    }
}
```